

PROJECT DOCUMENTATION PHASE

AI FAQ Assistant API

1. Introduction

The AI FAQ Assistant is a secure, RESTful backend platform that streamlines FAQ creation, user authentication, and AI-assisted content management. Built with Node.js and Express.js, it uses MongoDB (via Mongoose) as its data store and integrates Google Gemini AI to generate answers and structured FAQ pairs automatically, following the industry-standard MVC (Model-View-Controller) architecture.

Support teams typically manage large volumes of repetitive questions, and manually drafting, categorising, and searching this content is slow and error prone. This project addresses that gap by combining a secure CRUD API with generative AI, so FAQ entries can be authored, searched, and even auto generated through simple API calls.

1.1 Objectives

- Provide secure user registration and login using JWT-based authentication.
- Allow authenticated users to create, update, delete and manage their own FAQ records.
- Allow any client to publicly read and keyword-search published FAQs.
- Use Google Gemini AI to auto-generate direct answers and structured FAQ (question/answer) pairs.
- Enforce data integrity through Mongoose schema validation and centralised error handling.
- Protect user credentials using bcrypt password hashing.

1.2 Scope

The system is a headless backend (API-only) service. It exposes JSON REST endpoints that can be consumed by any client — a web front end, a mobile app, or an API testing tool such as Postman. The current scope covers two core resources, Users and FAQs, along with an AI service layer that augments FAQ content creation.

2. System Requirements

2.1 Software Requirements

Component	Specification
Operating System	Windows 10/11, macOS, or Linux
Runtime	Node.js v18 or above
Package Manager	npm v9 or above
Web Framework	Express.js 4.x
Database	MongoDB (local or MongoDB Atlas) with Mongoose ODM
AI Service	Google Gemini API (@google/genai SDK, gemini-2.0-flash / gemini-2.5-flash)
Authentication	JSON Web Token (jsonwebtoken) + bcrypt
API Testing Tool	Postman / VS Code REST client (Thunder Client)
Code Editor	Visual Studio Code

2.2 Hardware Requirements

Component	Minimum Specification
Processor	Intel Core i5 (8th Gen) / AMD Ryzen 5 or equivalent
RAM	8 GB (16 GB recommended)
Storage	1 GB free disk space
Internet	Required for Gemini AI API calls and MongoDB Atlas (if cloud-hosted)

3. System Architecture

The application follows a layered MVC architecture. Incoming HTTP requests pass through the Express server gateway, are filtered by middleware, handled by controllers, and finally interact with Mongoose models backed by MongoDB. AI-related requests are additionally routed through a dedicated service layer that calls the external Google Gemini API.

Layer	Responsibility
Client	Postman, web app, or mobile app sending HTTP/JSON requests.
Express Server (app.js / server.js)	Boots the HTTP server, parses JSON, applies CORS, and mounts routers.
Middleware	authMiddleware (JWT verification), validationMiddleware (request body checks), errorMiddleware (centralised error responses).
Routes	authRoutes, faqRoutes, aiRoutes — map URL paths to controller functions.
Controllers	authController, faqController, aiController — contain request-handling logic.
Services	geminiService — wraps the Google Gemini SDK for answer and FAQ generation.
Models	User.js, FAQ.js — Mongoose schemas defining validation rules and persistence.
Database	MongoDB collections: users, faqs.

3.1 MVC Breakdown

- **Model:** Mongoose schemas (User, FAQ) define field types, required rules, and defaults, and persist data to MongoDB.
- **Controller:** Express controller functions validate requests, orchestrate model/service calls, and return standardised JSON responses.
- **View:** Since this is a headless API, the "view" layer is the JSON response returned to the client rather than an HTML template.

4. Database Design

The database uses two primary MongoDB collections, connected through a foreign-key style reference.

4.1 User Collection

Field	Type	Rules
_id	ObjectId	Primary key (auto-generated)
name	String	Required
email	String	Required, unique, lowercase, validated format
password	String	Required, min length 6, hashed with bcrypt before save
createdAt / updatedAt	Date	Auto-managed timestamps

4.2 FAQ Collection

Field	Type	Rules
_id	ObjectId	Primary key (auto-generated)
question	String	Required
answer	String	Required
category	String	Required, enum: Technology, Education, Health, Banking, General (default: General)
createdBy	ObjectId (ref: User)	Required — links the FAQ to its creator
createdAt / updatedAt	Date	Auto-managed timestamps

4.3 Relationship

User → FAQ (One-to-Many): a single registered user can create and own many FAQ records. The FAQ document stores a createdBy reference back to its author, which is also used to verify ownership before allowing updates or deletes.

5. Module Description

- **Authentication Module:** Register (POST /api/auth/register) creates a user with a bcrypt-hashed password; Login (POST /api/auth/login) verifies credentials and issues a signed JWT; Profile (GET /api/auth/profile) returns the logged-in user's details (protected).
- **FAQ Management Module:** Create (POST /api/faqs, protected) adds a FAQ owned by the requester; Read (GET /api/faqs, GET /api/faqs/:id) is public; Update/Delete (PUT/DELETE /api/faqs/:id) are protected and restricted to the FAQ's creator; Search (GET /api/faqs/search?q=) performs public regex keyword search across question, answer and category.
- **AI Automation Module:** AI Answer (POST /api/ai/answer) sends a question to Gemini gemini-2.0-flash and returns a direct answer; AI FAQ Generator (POST /api/ai/generate-faq) sends a topic to Gemini gemini-2.5-flash using a structured JSON schema, returning a ready-to-store {question, answer} pair.

6. Technology Stack

Category	Technology
Runtime & Framework	Node.js, Express.js
Database	MongoDB, Mongoose ODM
Authentication & Security	JSON Web Token (jsonwebtoken), bcrypt
AI Integration	Google Gemini API (@google/genai)
Configuration	dotenv (.env environment variables)
Cross-Origin Support	cors
API Testing	Postman collection (AI_FAQ_Assistant_API.postman_collection.json)

7. Security Features

- Passwords are never stored in plain text — bcrypt hashes them with a salt before persistence.
- Protected routes require a valid JWT sent as an Authorization: Bearer <token> header.
- Only the original creator of a FAQ can update or delete it (ownership check in the controller).
- Centralised error middleware sanitises responses; validation middleware checks required fields before controller logic executes.

8. Advantages & Future Enhancements

- **Advantages:** Reduces manual FAQ drafting effort via AI; secure stateless authentication; clear MVC separation for easy maintenance; category-based organisation and keyword search improve discoverability.
- **Future Enhancements:** Role-based access control (Admin, Editor, Reader); semantic/embedding-based search; FAQ analytics (views, helpful/not-helpful feedback, tagging); a front-end web or mobile client.

9. Conclusion

The AI FAQ Assistant API successfully demonstrates a secure, well-structured backend that merges conventional CRUD operations with generative AI. Its layered MVC design, JWT-based security, and Gemini-powered content generation make it a practical, extensible foundation for real-world customer-support and knowledge-based applications.